

```
001  """Final individual prediction pipeline; run with `python submission_code.py`.
002
003  Requires train.csv and test_individual.csv beside this script. The recipe is
004  fixed. All fitting uses the 100,000 training rows; test labels are never used.
005  """
006
007  from __future__ import annotations
008
009  import gc
010  from pathlib import Path
011
012  import numpy as np
013  import pandas as pd
014  from lightgbm import LGBMClassifier, LGBMRegressor
015  from sklearn.compose import ColumnTransformer
016  from sklearn.impute import SimpleImputer
017  from sklearn.pipeline import Pipeline
018  from sklearn.preprocessing import OneHotEncoder
019
020
021  # Configuration
022  ROOT = Path(__file__).resolve().parent
023  TRAIN_PATH = ROOT / "train.csv"
024  TEST_PATH = ROOT / "test_individual.csv"
025  OUTPUT_PATH = ROOT / "submission_code_output.csv"
026  ID = "Id"
027  TARGETS = (
028      "Q0_TOTAL_ASSETS",
029      "Q0_TOTAL_LIABILITIES",
030      "Q0_TOTAL_STOCKHOLDERS_EQUITY",
031      "Q0_GROSS_PROFIT",
032      "Q0_COST_OF_REVENUES",
033      "Q0_REVENUES",
034      "Q0_OPERATING_INCOME",
035      "Q0_OPERATING_EXPENSES",
036      "Q0_EBITDA",
037  )
038  LAG_TARGETS = {"Q0_GROSS_PROFIT", "Q0_OPERATING_INCOME", "Q0_EBITDA"}
039  DIRECT_METHODS = {
040      "Q0_TOTAL_ASSETS": "large_leaf",
041      "Q0_TOTAL_LIABILITIES": "large_leaf",
042      "Q0_TOTAL_STOCKHOLDERS_EQUITY": "large_leaf",
043      "Q0_GROSS_PROFIT": "l1",
044      "Q0_COST_OF_REVENUES": "l1",
045      "Q0_OPERATING_INCOME": "sign_magnitude",
046      "Q0_OPERATING_EXPENSES": "large_leaf",
047      "Q0_EBITDA": "sign_magnitude",
048  }
049  THREADS = 4
050  SEED = 42
051
052
053  # Data loading
054  def load_data() -> tuple[pd.DataFrame, pd.DataFrame, list[str]]:
055      train = pd.read_csv(TRAIN_PATH, low_memory=False)
056      test = pd.read_csv(TEST_PATH, low_memory=False, dtype={ID: "string"})
057      if len(train) != 100_000 or len(test) != 50_000:
058          raise ValueError("Unexpected train or test row count")
059      if any(target not in train.columns for target in TARGETS):
```

```

060         raise ValueError("Training targets are missing")
061     if ID not in train.columns or ID not in test.columns:
062         raise ValueError("Id is missing")
063     predictors = [name for name in train.columns if name not in set(TARGETS) | {ID}]
064     if ID in predictors or set(predictors) & set(TARGETS):
065         raise AssertionError("Id or a Q0 target entered the predictors")
066     if predictors != [name for name in test.columns if name != ID]:
067         raise ValueError("Train/test predictor columns or their order differ")
068     if test[ID].isna().any() or not test[ID].is_unique:
069         raise ValueError("Test Id contains missing or duplicate values")
070     if not np.isfinite(train[list(TARGETS)].to_numpy(dtype=np.float64)).all():
071         raise ValueError("Training targets contain non-finite values")
072     return train, test, predictors
073
074
075 # Feature preprocessing
076 def make_matrices(train_rows: pd.DataFrame, test_rows: pd.DataFrame,
077                  predictors: list[str]) -> tuple[np.ndarray, np.ndarray]:
078     """Fit median imputation and one-hot categories on training rows only."""
079     if ID in predictors or set(predictors) & set(TARGETS):
080         raise AssertionError("Id or target entered the feature matrix")
081     numeric = [c for c in predictors if pd.api.types.is_numeric_dtype(train_rows[c])]
082     categorical = [c for c in predictors if c not in numeric]
083     processor = ColumnTransformer(
084         [
085             ("numeric", Pipeline([("impute", SimpleImputer(strategy="median"))]), numeric),
086             ("categorical", Pipeline([
087                 ("impute", SimpleImputer(strategy="most_frequent")),
088                 ("one_hot", OneHotEncoder(handle_unknown="ignore",
089                                         sparse_output=False, dtype=np.float32)),
090             ]), categorical),
091         ],
092         sparse_threshold=0.0,
093     )
094     x_train = processor.fit_transform(train_rows[predictors]).astype(np.float32, copy=False)
095     x_test = processor.transform(test_rows[predictors]).astype(np.float32, copy=False)
096     if not np.isfinite(x_train).all() or not np.isfinite(x_test).all():
097         raise ValueError("Preprocessed features contain non-finite values")
098     return x_train, x_test
099
100
101 # Lag feature engineering
102 def lag_features(rows: pd.DataFrame, target: str) -> pd.DataFrame:
103     """The exact eleven Q1-Q10 features used by the validated pipeline."""
104     suffix = target.removeprefix("Q0_")
105     values = np.column_stack([rows[f"Q{k}_{suffix}"].to_numpy(dtype=np.float64)
106                             for k in range(1, 11)])
107     if not np.isfinite(values).all():
108         raise ValueError(f"Non-finite lag values for {target}")
109     recent = values[:, :4]
110     weights4 = np.arange(3, -1, -1, dtype=np.float64) - 1.5
111     weights10 = np.arange(9, -1, -1, dtype=np.float64) - 4.5
112     stem = target.lower()
113     return pd.DataFrame({
114         f"{stem}_lag_q1": values[:, 0],
115         f"{stem}_q1_minus_q2": values[:, 0] - values[:, 1],
116         f"{stem}_q1_sign": np.sign(values[:, 0]),
117         f"{stem}_q2_sign": np.sign(values[:, 1]),
118         f"{stem}_q1_q2_same_sign": np.sign(values[:, 0]) * np.sign(values[:, 1]),

```

```

119         f"{stem}_q1_q4_mean": recent.mean(axis=1),
120         f"{stem}_q1_q4_median": np.median(recent, axis=1),
121         f"{stem}_q1_q4_std": recent.std(axis=1),
122         f"{stem}_q1_q4_slope_recent": recent @ weights4 / np.dot(weights4, weights4),
123         f"{stem}_q1_q10_mean": values.mean(axis=1),
124         f"{stem}_q1_q10_slope_recent": values @ weights10 / np.dot(weights10, weights10),
125     }, index=rows.index)
126
127
128 def matrices_for_target(train: pd.DataFrame, test: pd.DataFrame,
129                        base_predictors: list[str], target: str):
130     if target not in LAG_TARGETS:
131         return make_matrices(train, test, base_predictors)
132     train_lags = lag_features(train, target)
133     test_lags = lag_features(test, target)
134     if list(train_lags.columns) != list(test_lags.columns):
135         raise ValueError(f"Lag feature names differ for {target}")
136     train_rows = pd.concat([train, train_lags], axis=1)
137     test_rows = pd.concat([test, test_lags], axis=1)
138     return make_matrices(train_rows, test_rows,
139                        base_predictors + list(train_lags.columns))
140
141
142 # Target transformation
143 def target_scale(values: np.ndarray) -> float:
144     scale = float(np.median(np.abs(values)))
145     return scale if scale > 0 else 1.0
146
147
148 def arcsinh_prediction(model: LGBMRegressor, x_train: np.ndarray,
149                      y_train: np.ndarray, x_test: np.ndarray) -> np.ndarray:
150     scale = target_scale(y_train) # Only the full training target determines scale.
151     model.fit(x_train, np.arcsinh(y_train / scale))
152     return np.sinh(model.predict(x_test)) * scale
153
154
155 # LightGBM helpers
156 def regression_model(method: str) -> LGBMRegressor:
157     if method == "large_leaf":
158         config = {"num_leaves": 63, "min_child_samples": 20,
159                  "objective": "regression"}
160     elif method == "l1":
161         config = {"num_leaves": 31, "min_child_samples": 40,
162                  "objective": "regression_l1"}
163     else:
164         raise ValueError(f"Unknown direct regression method: {method}")
165     return LGBMRegressor(
166         **config, learning_rate=0.08, n_estimators=120,
167         colsample_bytree=0.8, n_jobs=THREADS, random_state=SEED,
168         deterministic=True, force_col_wise=True, verbosity=-1,
169     )
170
171
172 def sign_magnitude_model(*, classifier: bool):
173     cls = LGBMClassifier if classifier else LGBMRegressor
174     return cls(
175         n_estimators=120, learning_rate=0.08, num_leaves=31,
176         min_child_samples=40, colsample_bytree=0.8,
177         n_jobs=THREADS, random_state=SEED,

```

```

178         deterministic=True, force_col_wise=True, verbosity=-1,
179     )
180
181
182 # Sign + magnitude models
183 def predict_signed_magnitude(x_train: np.ndarray, y_train: np.ndarray,
184                             x_test: np.ndarray) -> np.ndarray:
185     classifier = sign_magnitude_model(classifier=True)
186     classifier.fit(x_train, (y_train > 0).astype(np.int8))
187     positive = classifier.predict_proba(x_test)[:, 1] >= 0.5
188     magnitude = arcsinh_prediction(
189         sign_magnitude_model(classifier=False), x_train, np.abs(y_train), x_test
190     )
191     return np.where(positive, magnitude, -magnitude)
192
193
194 # Train final direct models
195 def train_direct_models(train: pd.DataFrame, test: pd.DataFrame,
196                        predictors: list[str]) -> pd.DataFrame:
197     direct = pd.DataFrame(index=test.index, columns=TARGETS, dtype=np.float64)
198     base_x_train, base_x_test = matrices_for_target(
199         train, test, predictors, "Q0_TOTAL_ASSETS"
200     )
201     for target, method in DIRECT_METHODS.items():
202         if target in LAG_TARGETS:
203             x_train, x_test = matrices_for_target(train, test, predictors, target)
204         else:
205             x_train, x_test = base_x_train, base_x_test
206         y_train = train[target].to_numpy(dtype=np.float64)
207         if method == "sign_magnitude":
208             prediction = predict_signed_magnitude(x_train, y_train, x_test)
209         else:
210             prediction = arcsinh_prediction(
211                 regression_model(method), x_train, y_train, x_test
212             )
213         if not np.isfinite(prediction).all():
214             raise ValueError(f"Non-finite direct prediction for {target}")
215         direct[target] = prediction
216         print(f"trained {target}", flush=True)
217         if target in LAG_TARGETS:
218             del x_train, x_test
219         gc.collect()
220     return direct
221
222
223 # Accounting reconciliation
224 def reconcile_direct_predictions(direct: pd.DataFrame) -> pd.DataFrame:
225     """Every formula below reads only direct predictions, never final values."""
226     d_a = direct["Q0_TOTAL_ASSETS"]
227     d_l = direct["Q0_TOTAL_LIABILITIES"]
228     d_e = direct["Q0_TOTAL_STOCKHOLDERS_EQUITY"]
229     d_gp = direct["Q0_GROSS_PROFIT"]
230     d_c = direct["Q0_COST_OF_REVENUES"]
231     d_oi = direct["Q0_OPERATING_INCOME"]
232     d_oe = direct["Q0_OPERATING_EXPENSES"]
233     d_ebitda = direct["Q0_EBITDA"]
234     final = pd.DataFrame(index=direct.index)
235     final["Q0_TOTAL_ASSETS"] = d_a
236     final["Q0_TOTAL_LIABILITIES"] = 0.50 * d_l + 0.50 * (d_a - d_e)

```

```

237     final["Q0_TOTAL_STOCKHOLDERS_EQUITY"] = d_a - d_l
238     final["Q0_GROSS_PROFIT"] = d_gp
239     final["Q0_COST_OF_REVENUES"] = d_c
240     final["Q0_REVENUES"] = np.maximum(0.0, d_gp + d_c)
241     final["Q0_OPERATING_INCOME"] = 0.50 * d_oi + 0.50 * (d_gp - d_oe)
242     final["Q0_OPERATING_EXPENSES"] = np.maximum(
243         0.0, 0.25 * d_oe + 0.75 * (d_gp - d_oi)
244     )
245     final["Q0_EBITDA"] = d_ebitda
246     return final.loc[:, list(TARGETS)]
247
248
249 # Submission validation
250 def validate_submission(submission: pd.DataFrame, original_ids: pd.Series) -> None:
251     if submission.shape != (50_000, 10) or list(submission.columns) != [ID, *TARGETS]:
252         raise ValueError("Submission shape or column order differs from rubric")
253     if not submission[ID].equals(original_ids):
254         raise ValueError("Test Id values or order changed")
255     if submission[ID].isna().any() or not submission[ID].is_unique:
256         raise ValueError("Submission Id has missing or duplicate values")
257     if not np.isfinite(submission[list(TARGETS)].to_numpy(dtype=np.float64)).all():
258         raise ValueError("Submission contains NaN or infinite predictions")
259     if (submission["Q0_REVENUES"] < 0).any():
260         raise ValueError("Revenue predictions are negative")
261     if (submission["Q0_OPERATING_EXPENSES"] < 0).any():
262         raise ValueError("Operating expense predictions are negative")
263
264
265 # Save predictions
266 def main() -> None:
267     train, test, predictors = load_data()
268     direct = train_direct_models(train, test, predictors)
269     final = reconcile_direct_predictions(direct)
270     submission = pd.concat([test[[ID]], final], axis=1)
271     validate_submission(submission, test[ID])
272     submission.to_csv(OUTPUT_PATH, index=False)
273     saved = pd.read_csv(OUTPUT_PATH, low_memory=False, dtype={ID: "string"})
274     validate_submission(saved, test[ID])
275     print(f"saved {OUTPUT_PATH} ({len(saved):,} rows)", flush=True)
276
277
278 if __name__ == "__main__":
279     main()

```